American International University-Bangladesh (AIUB)

Faculty of Science and Technology

Department of Computer Science

Web Technologies

CSE 4101 / [Course Code]

PROJECT REPORT

Project Title:

Travel Guide

Semester: [Spring 2025–2026]
Submission Date: [May-18-2026]
Section: [L]

Submitted To:

[WAHIDUL ALAM RIYAD]
[CSE]
American International University-Bangladesh

Group Information

Group Number: 08

Project Title: Travel Guide

Student Information:

Full Name: Sunjeta Jahan Snegdha

Student ID: 23-52691-2

Program: CSE

Email: snegdha171119@gmail.com

Signature: _____

Declaration: We hereby declare that this project report is our original work. Any external resources or references used have been properly cited. We have not engaged in any form of academic dishonesty.

Marks Sheet

To be filled by the instructor only. Students must NOT write in this section.

Group No.		**Section**	
Member 1 ID			

CO3: Illustrate Multi-Tier Web Application for Targeted Society					
Feature Implementation (0-3)			**Feature Implementation (0-3)**		**Completeness (0-3)**
Basic Web Security	**UI (HTML/CSS)**	**Feature Completeness**	**DB**	**Auth (Session/cookie)**	**MVC**
CO4: Coordinate with Group Members to Develop Multi-Tier Application					
Technical Knowledge (0-3)			**Collaborative Teamwork (0-3)**		**Communication/Promptness (0-3)**

JS validation	PHP Validation	Ajax/JSON	Git Contribution	VIVA
Expected Outcome (0-2)		**CO3 Total**	**CO4 Total**	**Project Total**

Evaluated By: _____ **Date:** _____

Course Teacher Signature: _____

Table of Contents

Chapter 1: Project Overview

1.1 Project Description

Travel Guide is a web-based application that helps visitors from different countries get suggestions and preferences for selecting visiting places worldwide. The website is enriched with information about sight-seeing and visiting places around the globe.

The system supports three user roles:

- ADMIN — Main controlling point who adds/removes users, makes posts available, and removes comments.
- SCOUT — Collects information about visiting places and creates post requests for admin to publish.
- General USER — Views published posts, maintains a wishlist, gets probable cost information, and posts comments.

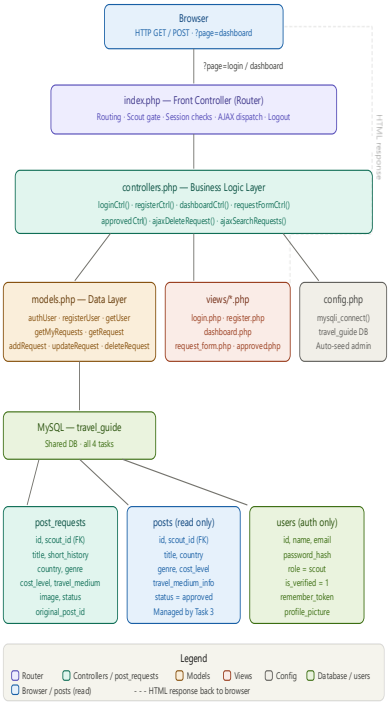
My role (Task 2) covers the Scout dashboard, post request creation with image upload, editing and deleting pending requests, viewing approved posts, and requesting changes to approved posts.

1.2 Technologies Used

List all technologies, frameworks, and tools used in this project.

Layer	Technology	Purpose
Frontend	HTML5, CSS3	UI Structure & Styling
Frontend	JavaScript (Vanilla)	Client-side validation, AJAX delete & search
Backend	PHP (Procedural)	Server-side logic & routing
Database	MySQL / MariaDB	Data storage
Auth	PHP Sessions & Cookies	Login management & Remember Me
MVC	Custom MVC (flat)	Separation of concerns
Version Control	Git / GitHub	Collaboration & versioning
Server	XAMPP / Apache	Local development environment

1.3 System Architecture / MVC Diagram



1.4 Database Schema

Table	Field Name	Data Type	Description
users	id	INT AUTO_INCREMENT	Primary Key
users	name	VARCHAR(100)	Full name
users	email	VARCHAR(150) UNIQUE	Login email
users	password_hash	VARCHAR(255)	Bcrypt hashed password
users	role	ENUM(admin,scout,user)	User role
users	is_verified	TINYINT(1)	Admin verification status
users	profile_picture	VARCHAR(255)	Path to uploaded image
users	remember_token	VARCHAR(255)	Hashed Remember Me token
users	created_at	DATETIME	Registration timestamp

wishlist	id	INT AUTO_INCREMENT	Primary Key
wishlist	user_id	INT (FK)	References users.id
wishlist	post_id	INT (FK)	References posts.id
wishlist	added_at	DATETIME	When post was saved

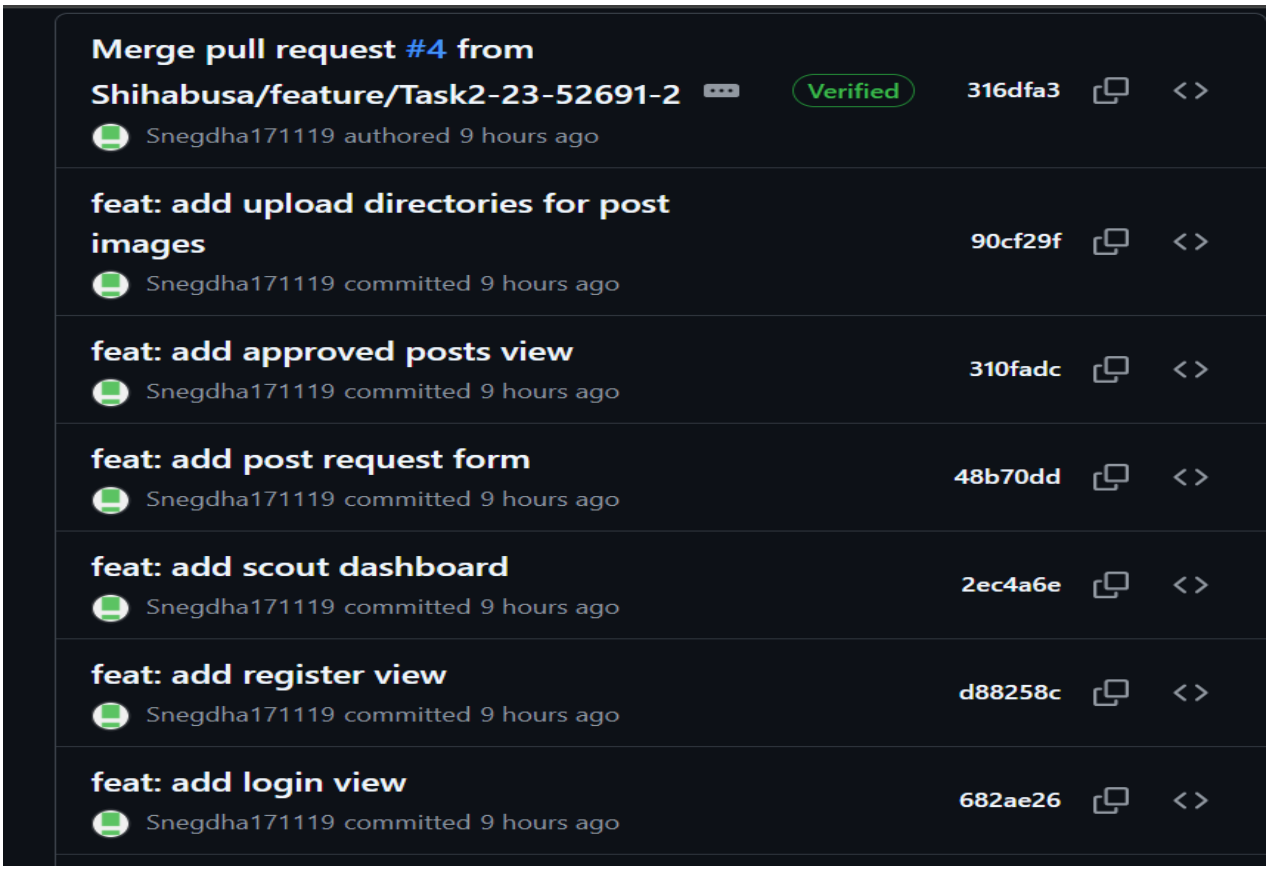
Chapter 2: Git Contribution & Task Allocation

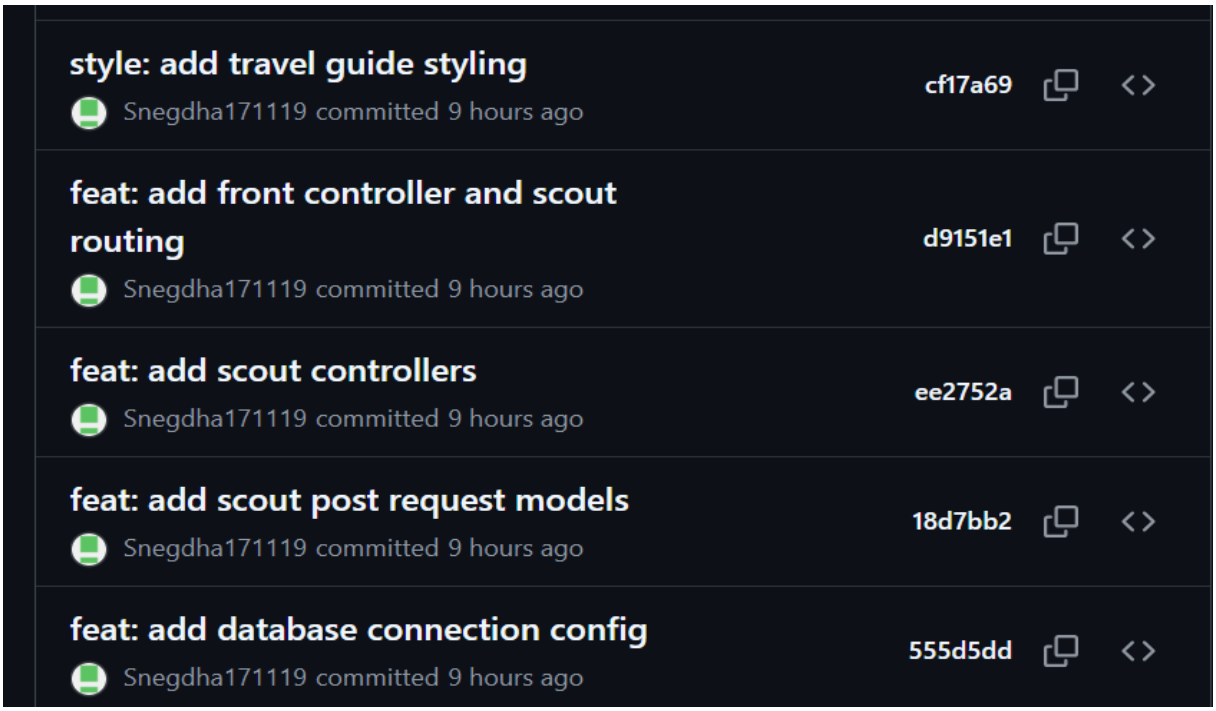
2.1 GitHub Repository Link

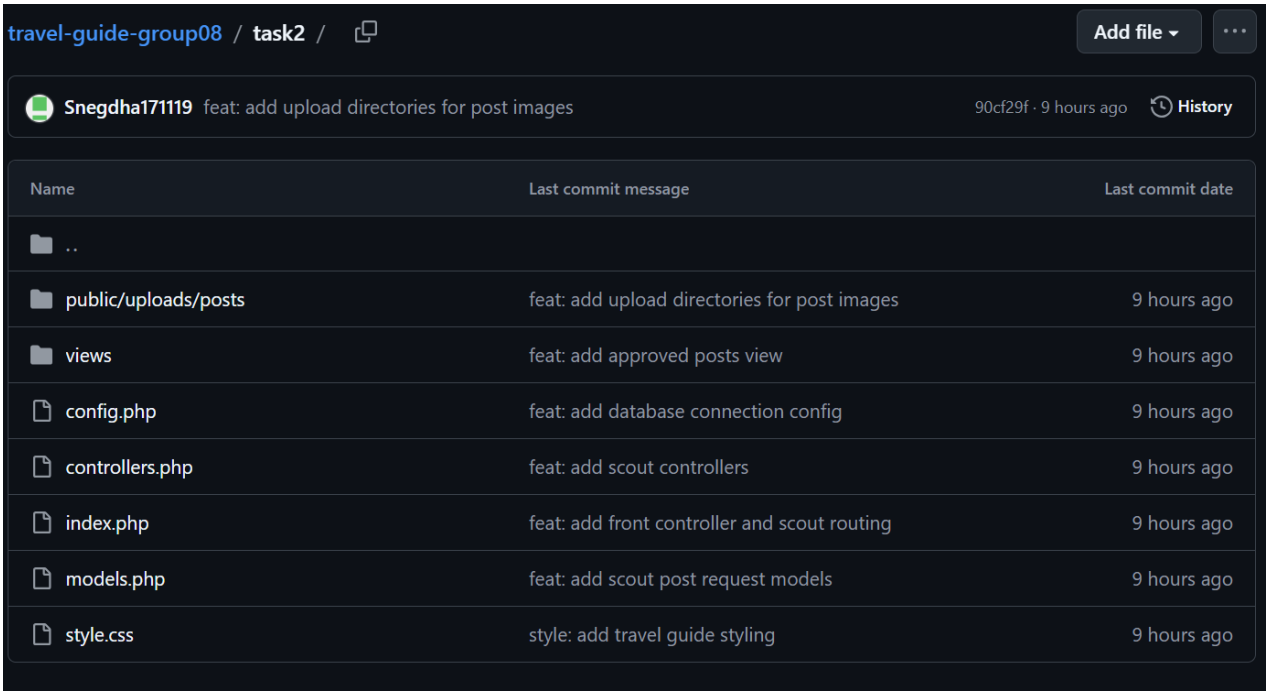
I used feature branches and Pull Requests as per the Git workflow requirements. My commits were made on the branch feature/Task2-23-52691-2 and merged into main via a Pull Request.

Repository URL: https://github.com/Shihabusa/travel-guide-group08

2.2 Git Contribution Screenshot




style: add travel guide styling
Snegdha171119 committed 9 hours ago
cf17a69
feat: add front controller and scout routing
Snegdha171119 committed 9 hours ago
d9151e1
feat: add scout controllers
Snegdha171119 committed 9 hours ago
ee2752a
feat: add scout post request models
Snegdha171119 committed 9 hours ago
18d7bb2
feat: add database connection config
Snegdha171119 committed 9 hours ago
555d5dd


travel-guide-group08 / task2 /
Add file
Snegdha171119 feat: add upload directories for post images
90cf29f · 9 hours ago
History
Name
Last commit message
Last commit date
..
public/uploads/posts
feat: add upload directories for post images
9 hours ago
views
feat: add approved posts view
9 hours ago
config.php
feat: add database connection config
9 hours ago
controllers.php
feat: add scout controllers
9 hours ago
index.php
feat: add front controller and scout routing
9 hours ago
models.php
feat: add scout post request models
9 hours ago
style.css
style: add travel guide styling
9 hours ago

Chapter 3: Features, Source Code & Screenshots

For each feature implemented in the project, provide the following: a description, the relevant source code with explanation, and screenshot(s) of the UI.

Duplicate the feature section (Feature 1 block) as many times as needed — one block per feature.

Feature 1: [Scout Gate & Authentication]

Description

Only users with role = 'scout' AND is_verified = 1 can access scout pages. If a non-scout user or unverified scout tries to access any scout page, they are immediately redirected to the login page. Login creates session variables and supports Remember Me with a 30-day cookie.

Relevant Source Code

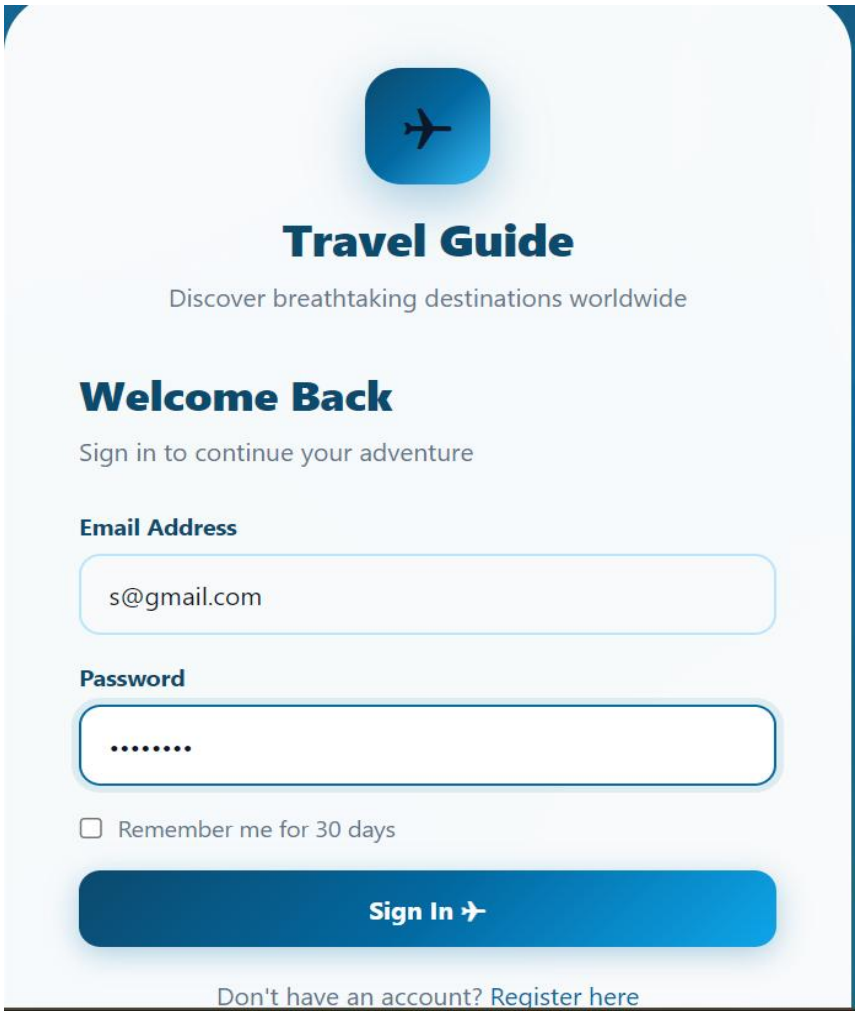
File: /index.php — Scout gate

```php
$scoutPages = ['dashboard', 'request_form', 'approved'];
if (in_array($page, $scoutPages)) {
    if ($_SESSION['user']['role'] !== 'scout') {
        header('Location: index.php?page=login');
        exit;
    }
    if (!$_SESSION['user']['is_verified']) {
    $_SESSION = [];
    session_destroy();
    header('Location: index.php?page=login');
    exit;
}
}
```

Code Explanation

- Scout gate checks three conditions: user logged in, role = scout, is_verified = 1.
- If any condition fails, session is destroyed and user is redirected to login.
- This prevents unauthorized access to all scout pages.
- Remember Me stores SHA-256 hashed token in DB and 30-day cookie for auto-login.

Screenshot(s)



Feature 2: Create Post Request with Image Upload

Description

Scouts can submit new post requests containing title, short history, country, genre, cost level, travel medium info, and an optional image. Data is saved to the post_requests table with status = pending. Images are validated for MIME type (JPG/PNG/GIF/WEBP) and size (max 2MB) before upload to public/uploads/posts/.

Relevant Source Code

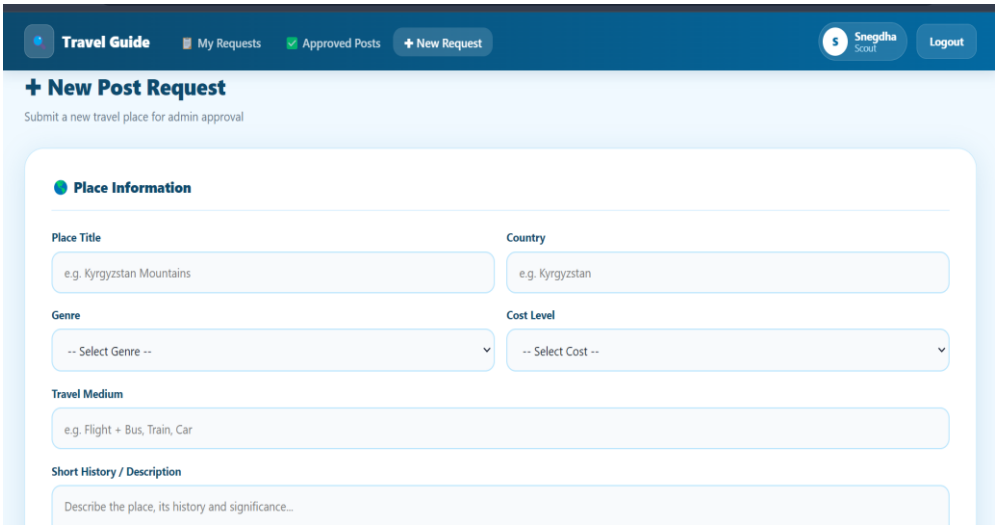
File: /controllers.php — requestFormCtrl() add section

```php
/* Add */
if ($action === 'add' && $_SERVER['REQUEST_METHOD'] === 'POST') {
    $title         = trim($_POST['title'] ?? '');
    $shortHistory  = trim($_POST['short_history'] ?? '');
    $country       = trim($_POST['country'] ?? '');
    $genre         = $_POST['genre'] ?? '';
    $costLevel     = $_POST['cost_level'] ?? '';
    $travelMedium  = trim($_POST['travel_medium_info'] ?? '');
    $originalPostId = intval($_POST['original_post_id'] ?? 0) ?: null;
    $image         = null;

    // Validation
    if ($title === '' || $shortHistory === '' || $country === '' || $travelMedium === '') {
        $error = 'All fields are required.';
    } elseif (!in_array($genre, ['beach','mountain','city','historical','nature','other'])) {
        $error = 'Please select a valid genre.';
    } elseif (!in_array($costLevel, ['low','medium','high'])) {
        $error = 'Please select a valid cost level.';
    } else {
        // Handle image upload
        if (!empty($_FILES['image']['name'])) {
            $file    = $_FILES['image'];
            $allowed = ['image/jpeg', 'image/png', 'image/gif', 'image/webp'];
            $maxSize = 2 * 1024 * 1024;

            if (!in_array($file['type'], $allowed)) {
                $error = 'Only JPG, PNG, GIF or WEBP images allowed.';
            } elseif ($file['size'] > $maxSize) {
                $error = 'Image must be under 2MB.';
            } else {
                $ext   = pathinfo($file['name'], PATHINFO_EXTENSION);
                $fname = 'post_' . $scoutId . '_' . time() . '.' . $ext;
                $dest  = 'public/uploads/posts/' . $fname;
                if (move_uploaded_file($file['tmp_name'], $dest)) {
                    $image = $dest;
                } else {
                    $error = 'Failed to upload image.';
                }
            }
        }
```

Code Explanation

- All text fields validated server-side before any DB write.
- in_array() checks MIME type against whitelist to prevent malicious file uploads.
- File size checked against 2MB limit before moving to uploads directory.
- Unique filename generated with timestamp to prevent filename collisions.
- move_uploaded_file() safely transfers temp file to public/uploads/posts/.
- addRequest() uses prepared statements to prevent SQL injection.
- original_post_id field supports change requests for approved posts.

Screenshot(s)



Feature 3: My Requests Dashboard with AJAX Search

Description

The dashboard shows all post requests made by the logged-in scout in a searchable table. Each row shows title, country, genre, cost level, status badge, and submission date. Edit and Delete buttons appear only for pending requests. A live AJAX search filters results as the user types without page reload.

Relevant Source Code

File: /views/dashboard.php — AJAX search

```
input.addEventListener('input', function () {
    clearTimeout(timer);
    timer = setTimeout(function () {
        fetch('index.php?page=ajax&type=search_requests&q=' + encodeURIComponent(input.value.trim()),
            { credentials: 'same-origin' })
            .then(function (r) { return r.json(); })
            .then(render)
            .catch(function (e) { console.error(e); });
    }, 200);
});
```

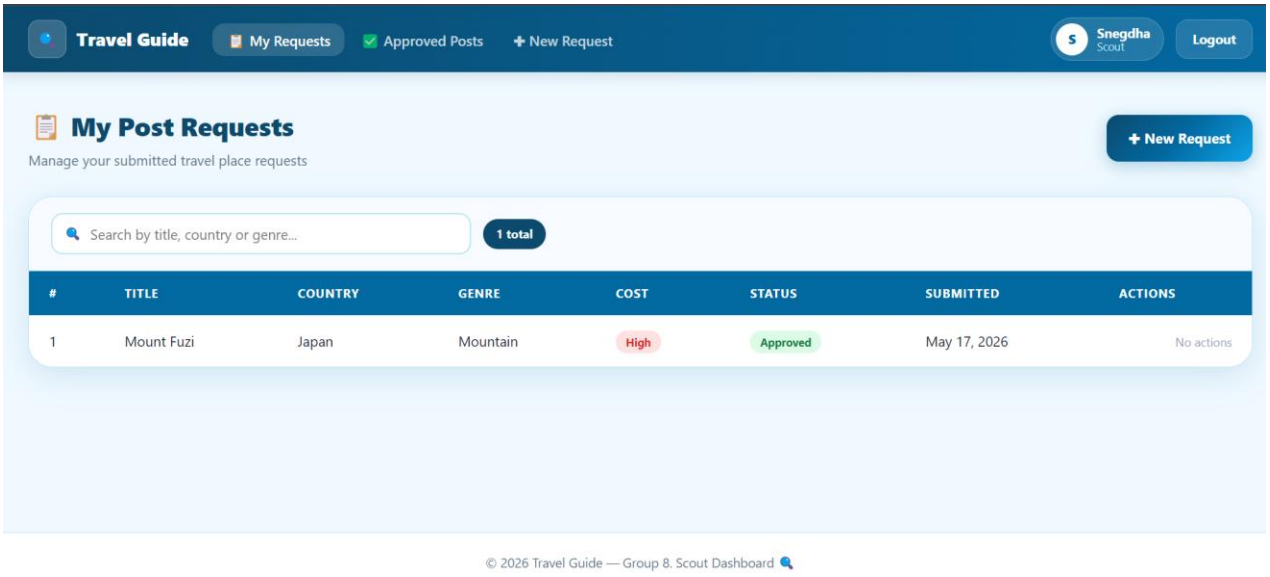
File: /models.php — searchMyRequests()

```php
/* Search Requests */
function searchMyRequests($conn, $scoutId, $term) {
    $like = '%' . $term . '%';
    $stmt = mysqli_prepare($conn,
        "SELECT id, title, country, genre, cost_level, status, requested_at
         FROM post_requests
         WHERE scout_id = ? AND (title LIKE ? OR country LIKE ? OR genre LIKE ?)
         ORDER BY requested_at DESC");
    mysqli_stmt_bind_param($stmt, 'isss', $scoutId, $like, $like, $like);
    mysqli_stmt_execute($stmt);
    $rows = mysqli_fetch_all(mysqli_stmt_get_result($stmt), MYSQLI_ASSOC);
    mysqli_stmt_close($stmt);
    return $rows;
}
?>
```

Code Explanation

- 200ms debounce delay prevents excessive AJAX calls on every keystroke.
- encodeURIComponent() safely encodes search query for URL transmission.
- AJAX endpoint returns JSON array of matching requests.
- scout_id = ? ensures scouts only see their own requests (data isolation).
- LIKE % wildcard searches across title, country and genre fields.
- Prepared statements prevent SQL injection in search queries.

Screenshot(s)



Feature 4: Edit & Delete Pending Requests (AJAX Delete)

Description

Scouts can edit or delete their own pending requests. Edit loads the request form pre-filled with existing data. Delete uses AJAX to remove the request without page reload, fading out the row from the table. Only pending requests show Edit/Delete buttons — approved and rejected requests are read-only.

Relevant Source Code

File: /views/dashboard.php — AJAX delete

```
function bindDeleteButtons() {
    document.querySelectorAll('.delete-btn').forEach(function (btn) {
        btn.addEventListener('click', function (e) {
            e.preventDefault();
            if (!confirm('Delete this request? This cannot be undone.')) return;

            var id    = this.getAttribute('data-id');
            var rowId = this.getAttribute('data-row');
            var self  = this;

            self.textContent = 'Deleting...';
            self.style.pointerEvents = 'none';

            fetch('index.php?page=ajax&type=delete_request', {
                method: 'POST',
                credentials: 'same-origin',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ id: parseInt(id) })
            })
            .then(function (r) { return r.json(); })
            .then(function (data) {
                if (data.success) {
                    var row = document.getElementById(rowId);
                    if (row) {
                        row.style.opacity = '0';
                        row.style.transition = 'opacity .3s ease';
                        setTimeout(function () { row.remove(); }, 300);
                    }
                    var cnt = document.getElementById('resultCount');
                    if (cnt) {
                        var num = parseInt(cnt.textContent) - 1;
                        cnt.textContent = num + ' total';
                    }
                } else {
                    alert(data.error || 'Failed to delete.');
                    self.textContent = 'Delete';
                    self.style.pointerEvents = '';
                }
            })
            .catch(function () {
                alert('Something went wrong.');
                self.textContent = 'Delete';
                self.style.pointerEvents = '';
```

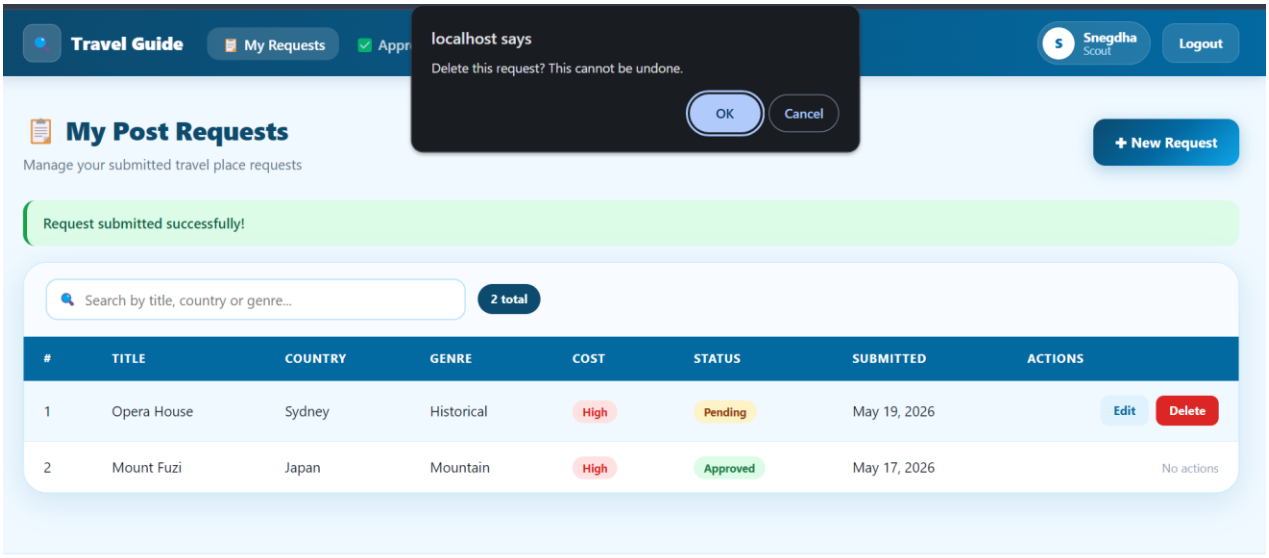
File: /controllers.php — ajaxDeleteRequest()

```php
/* AJAX Delete Request  */
function ajaxDeleteRequest($conn) {
    header('Content-Type: application/json');

    if (!isset($_SESSION['user'])) {
        http_response_code(403);
        echo json_encode(['error' => 'Unauthorized']);
        exit;
    }

    $data    = json_decode(file_get_contents('php://input'), true);
    $id      = intval($data['id'] ?? 0);
    $scoutId = $_SESSION['user']['id'];

    if ($id <= 0) {
        http_response_code(400);
        echo json_encode(['error' => 'Invalid request ID.']);
        exit;
    }

    if (deleteRequest($conn, $id, $scoutId)) {
        echo json_encode(['success' => true, 'message' => 'Request deleted successfully.']);
    } else {
        http_response_code(500);
        echo json_encode(['error' => 'Failed to delete. Only pending requests can be deleted.']);
    }
    exit;
}
```

Code Explanation

- confirm() prompts user before deletion to prevent accidental deletes.
- AJAX POST sends delete request without page reload.
- Ownership check ensures scouts can only delete their own requests.
- Status check prevents deleting approved or rejected requests.
- Row fade-out animation provides smooth visual feedback on delete.
- http_response_code(403) returns proper HTTP status for forbidden actions.

Screenshot(s)



Feature 5: Approved Posts & Request Changes

Description

Scouts can view which of their submissions have been approved by admin. Approved posts are read from the posts table (managed by Task 3). For each approved post, a 'Request Changes' button allows the scout to submit a change request. This creates a new post_request with the original_post_id field linking to the approved post.

Relevant Source Code

File: /models.php — getMyApprovedPosts()

```php
/*  Approved Posts */
function getMyApprovedPosts($conn, $scoutId) {
    $stmt = mysqli_prepare($conn,
        "SELECT id, title, country, genre, cost_level, travel_medium_info, image, created_at
         FROM posts WHERE scout_id = ? AND status = 'approved'
         ORDER BY created_at DESC");
    mysqli_stmt_bind_param($stmt, 'i', $scoutId);
    mysqli_stmt_execute($stmt);
    $rows = mysqli_fetch_all(mysqli_stmt_get_result($stmt), MYSQLI_ASSOC);
    mysqli_stmt_close($stmt);
    return $rows;
}

function getApprovedPost($conn, $id) {
    $stmt = mysqli_prepare($conn,
        "SELECT * FROM posts WHERE id = ? AND status = 'approved'");
    mysqli_stmt_bind_param($stmt, 'i', $id);
    mysqli_stmt_execute($stmt);
    $row = mysqli_fetch_assoc(mysqli_stmt_get_result($stmt));
    mysqli_stmt_close($stmt);
    return $row;
}
```

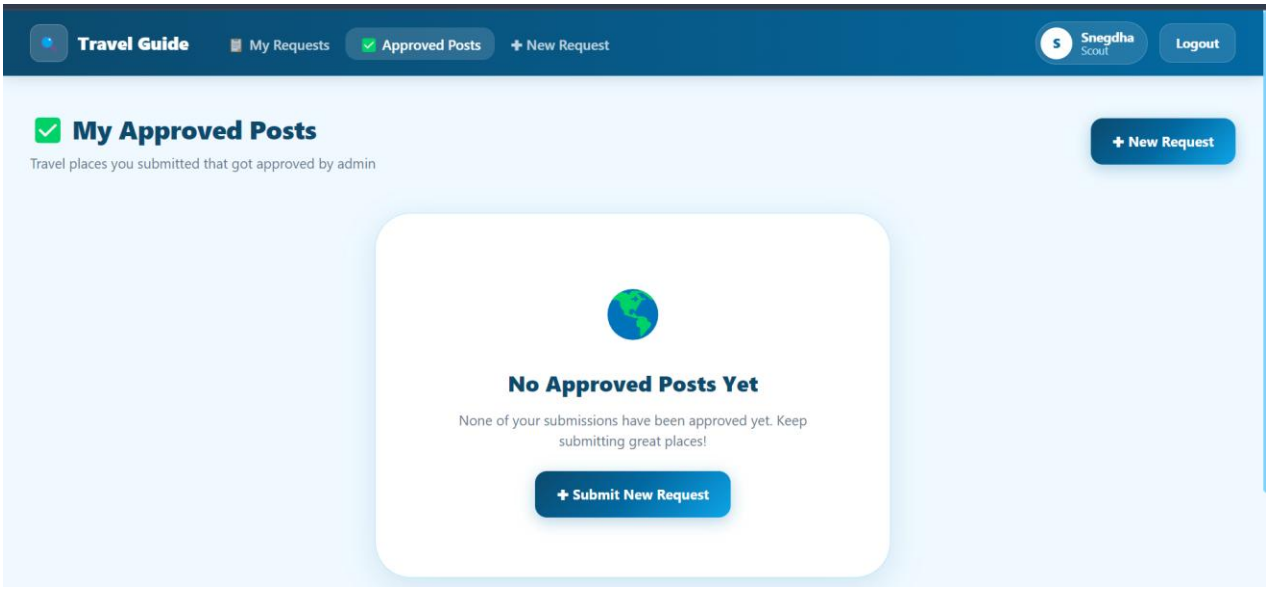
File: /views/approved.php — Request Changes button

```php
<!-- Request changes button -->
<div style="margin-top:12px;">
    <a href="index.php?page=request_form&action=add&original_post_id=<?= $post['id'] ?>"
       class="btn btn-ghost"
       style="font-size:12px; padding: 8px 14px;">
        &#9998; Request Changes
    </a>
</div>
```

Code Explanation

- scout_id = ? ensures scouts only see their own approved posts.
- status = 'approved' filters only admin-approved posts from posts table.
- Request Changes passes original_post_id as URL parameter to request form.
- New post_request is saved with original_post_id linking to approved post.
- Admin (Task 3) reviews the change request like any other pending request.

Screenshot(s)



Feature 6: JS Validation on Request Form

Description

The post request form includes comprehensive JavaScript validation. All required fields are validated on input (live validation) and on form submission. Error messages appear inline below each field. Image files are validated for type and size before upload. The form cannot be submitted with invalid data.

Relevant Source Code

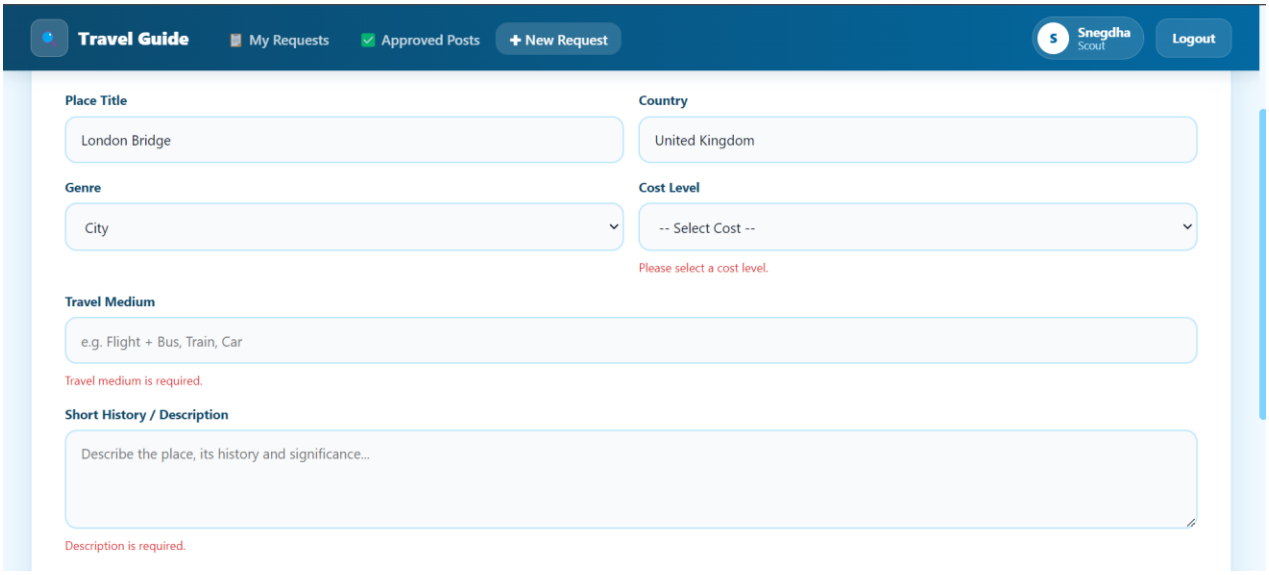
File: /views/request_form.php — JS validation

```
/* Image validation */
image.addEventListener('change', function () {
    if (image.files.length > 0) {
        var file    = image.files[0];
        var allowed = ['image/jpeg', 'image/png', 'image/gif', 'image/webp'];
        var maxSize = 2 * 1024 * 1024;
        if (!allowed.includes(file.type)) {
            alert('Only JPG, PNG, GIF or WEBP images are allowed.');
            image.value = '';
        } else if (file.size > maxSize) {
            alert('Image must be under 2MB.');
            image.value = '';
        }
    }
});

/* Submit validation */
form.addEventListener('submit', function (e) {
    var valid = true;

    if (title.value.trim() === '') {
        showErr(titleErr, 'Title is required.'); valid = false;
    }
    if (country.value.trim() === '') {
        showErr(countryErr, 'Country is required.'); valid = false;
    }
    if (genre.value === '') {
        showErr(genreErr, 'Please select a genre.'); valid = false;
    }
    if (cost.value === '') {
        showErr(costErr, 'Please select a cost level.'); valid = false;
    }
    if (travel.value.trim() === '') {
        showErr(travelErr, 'Travel medium is required.'); valid = false;
    }
    if (history.value.trim() === '') {
        showErr(historyErr, 'Description is required.'); valid = false;
    } else if (history.value.trim().length < 20) {
        showErr(historyErr, 'Description must be at least 20 characters.'); valid = false;
    }

    if (!valid) e.preventDefault();
});
```

Code Explanation

- e.preventDefault() stops form submission if any field fails validation.
- Live 'input' event validation gives immediate feedback as user types.
- Description minimum length of 20 characters ensures meaningful content.
- Image MIME type validated client-side before server upload.
- showErr() and clearErr() functions manage inline error message display.

Screenshot(s)



Chapter 5: Conclusion & Future Work

5.1 Summary

Task 2 of the Travel Guide project covered the complete Scout dashboard including post request creation with image upload, editing and deleting pending requests, AJAX search, viewing approved posts, and requesting changes to approved posts. All 10 grading criteria were successfully implemented including Basic Web Security, UI/CSS, Feature Completeness, Database, Auth (Session/Cookie), MVC, JS Validation, PHP Validation, AJAX/JSON, and Git Contribution.

The system uses procedural PHP with mysqli prepared statements throughout, ensuring SQL injection prevention. Image uploads are validated for MIME type and size. All forms have both client-side and server-side validation. AJAX endpoints return proper JSON responses with appropriate HTTP status codes.

5.2 Challenges Faced

- Redirect Loop: After login, unverified scouts caused infinite redirects between dashboard and login. Fixed by destroying session and redirecting cleanly to login without msg=not_verified parameter.
- Image Upload Path: The public/uploads/posts/ directory did not exist causing upload failures. Fixed by manually creating the directory structure on deployment.

- AJAX Delete Ownership: Initial implementation did not verify request ownership before deletion. Fixed by adding scout_id check in ajaxDeleteRequest() to prevent cross-user deletions.
- Approved Posts Empty: Approved posts page was empty because post_requests status was changed manually instead of data being moved to posts table. This is Task 3's responsibility.

5.3 Future Improvements

- Email Notifications: Send email to scout when admin approves or rejects their request.
- Draft Mode: Allow scouts to save requests as drafts before submitting for admin review.
- Multiple Images: Support uploading multiple images per post request with a gallery view.
- Rich Text Editor: Replace plain textarea with a rich text editor for short history field.
- Request History: Show full history of changes and status updates for each post request.
- Admin Comments: Allow admin to add rejection reason when rejecting a request

References

[1] Author, A. (Year). Title of resource. Source URL.

[2] Author, B. (Year). Title of resource. Source URL.

[3] [Add more as needed]